

ECA1418 - Embedded Systems

Analyze the embedded Software testing Strategy for an automotive ECU that experienced a Safety-critical race condition during field testing - two CAN messages arriving within 500 μ s caused a vehicle stall at 80 km/h. Examine which testing level (Unit, Integration, HIL, System) should have detected this defect, identify the coverage metric gap (MC/DC) calculate the cost of detecting this bug at the current phase vs. design review, and redesign the testing process to structurally prevent such escape defects.

1. which Testing Level should have detected the defect?

Testing Level	Purpose
unit testing	Tests individual functions / modules
Integration testing ✓	Tests communication between software modules
Hardware-in-the-loop testing	Tests ECU with simulated vehicle hardware and real time signals.
System testing	Tests the complete vehicle

The defect should ideally have been detected during integration testing or HIL Testing, with HIL being the most effective level because it can accurately simulate real-time CAN traffic.

2. Coverage Metric Map (MC/DC)

MC/DC is required by ISO 26262 for Safety-critical automotive software

Suppose the ECU contains logic like:

```
if (Engine-Run &&  
    CAN-Message 1 &&  
    CAN-Message 2 &&  
    ! Fault)
```

```
{  
    Fuel-Enable();
```

```
}
```

Normal testing may only execute:

- Message 1 First
- Message 2 first
- single Message

it may never test:

- Both CAN messages arriving within 500µs.

Therefore:

- Decision executed
- Every condition TRUE/FALSE
- Timing interaction NOT tested

Coverage Gap

MC/DC measures logical decision coverage but does not guarantee timing or concurrency coverage

Additional testing needed:

- interrupt timing tests
- Race-condition tests
- stress testing
- Boundary timing tests

3. Cost of Detecting the Bug

A commonly used Software engineering rule is that fixing a defect becomes about 10x more expensive at each later phase.

Phase

Relative cost

Design Review

1x

Coding

10x

integration testing

100x

System testing

1000x

Field Testing

10,000x

Example calculation

Assume:

- cost during design review = ₹5,000

Then:

- Field testing cost = ₹5,000 × 10,000 = 50,000,000

Extra cost due to late detection:

$$₹50,000,000 - ₹5,000 = ₹4,99,95,000$$

This excludes additional costs such as recalls, warranty claims, customer dissatisfaction, and safety investigations.

4. Redesigned testing Process

Step 1: Design review

- Analyze concurrent CAN message handling
- Review interrupt priorities.
- identify race conditions using design analysis.

Step 2: unit testing

- Test every software module
- use boundary-value and error-condition tests
- verify interrupt-safe functions.
- Aim for 100% MC/DC on safety-critical code.

Step 3: integration Testing

- inject multiple CAN messages simultaneously
- validate interrupt interactions
- include stress and load tests

Step 5: System testing

- Test the complete vehicle
- validate fail-safe behaviour
- Perform high-speed driving scenarios.

5. Structural improvements to prevent similar defect

- use MC/DC for all safety-critical software
- Add timing and concurrency test cases.
- Perform static analysis to identify race conditions
- uses automated CAN traffic generation with microsecond-level timing.
- perform stress and endurance testing
- Conduct design reviews focused on interrupt handling.
- Follow ISO 26262 Safety Processes and maintain traceability from requirements to test cases.